Use the API if single sign-on is enabled

StorageGRID 11.7

NetApp
March 05, 2024

This PDF was generated from https://docs.netapp.com/us-en/storagegrid-117/admin/use-api-if-sso-is-enabled-adfs.html on March 05, 2024. Always check docs.netapp.com for the latest.

Table of Contents

Use the API if single sign-on is enabled 1
Use the API if single sign-on is enabled (Active Directory) 1
Use the API if single sign-on is enabled (Azure) 8
Use the API if single sign-on is enabled (PingFederate) 9

Use the API if single sign-on is enabled

Use the API if single sign-on is enabled (Active Directory)

If you have configured and enabled single sign-on (SSO) and you use Active Directory as the SSO provider, you must issue a series of API requests to obtain an authentication token that is valid for the Grid Management API or the Tenant Management API.

Sign in to the API if single sign-on is enabled

These instructions apply if you are using Active Directory as the SSO identity provider.

Before you begin

- You know the SSO username and password for a federated user who belongs to a StorageGRID user group.
- If you want to access the Tenant Management API, you know the tenant account ID.

About this task

To obtain an authentication token, you can use one of the following examples:

- The `storagegrid-ssoauth.py` Python script, which is located in the StorageGRID installation files directory (`./rpms` for Red Hat Enterprise Linux or CentOS, `./debs` for Ubuntu or Debian, and `./vsphere` for VMware).
- An example workflow of curl requests.

  The curl workflow might time out if you perform it too slowly. You might see the error: `A valid SubjectConfirmation was not found on this Response`.

  The example curl workflow does not protect the password from being seen by other users.

If you have a URL-encoding issue, you might see the error: `Unsupported SAML version`.

Steps

1. Select one of the following methods to obtain an authentication token:
   - Use the `storagegrid-ssoauth.py` Python script. Go to step 2.
   - Use curl requests. Go to step 3.
2. If you want to use the `storagegrid-ssoauth.py` script, pass the script to the Python interpreter and run the script.

   When prompted, enter values for the following arguments:

   - The SSO method. Enter ADFS or adfs.
   - The SSO username
   - The domain where StorageGRID is installed
   - The address for StorageGRID

- The tenant account ID, if you want to access the Tenant Management API.

```
python3 storagegrid-ssoauth.py
sso_method: adfs
saml_user: my-sso-username
saml_domain: my-domain
sg_address: storagegrid.example.com
tenant_account_id: 12345
Enter the user's SAML password:
******************************************************************************

******************************************************************************
StorageGRID Auth Token: 56eb07bf-21f6-40b7-afob-5c6cacfb25e7
```

The StorageGRID authorization token is provided in the output. You can now use the token for other requests, similar to how you would use the API if SSO was not being used.

3. If you want to use curl requests, use the following procedure.
   a. Declare the variables needed to sign in.

```
export SAMLUSER='my-sso-username'
export SAMLPASSWORD='my-password'
export SAMLDOMAIN='my-domain'
export TENANTACCOUNTID='12345'
export STORAGEGRID_ADDRESS='storagegrid.example.com'
export AD_FS_ADDRESS='adfs.example.com'
```

> To access the Grid Management API, use 0 as `TENANTACCOUNTID`.

   b. To receive a signed authentication URL, issue a POST request to `/api/v3/authorize-saml`, and remove the additional JSON encoding from the response.

This example shows a POST request for a signed authentication URL for `TENANTACCOUNTID`. The results will be passed to `python -m json.tool` to remove the JSON encoding.

```
curl -X POST "https://$STORAGEGRID_ADDRESS/api/v3/authorize-saml" \
  -H "accept: application/json" -H  "Content-Type: application/json"
\
  --data "{\"accountId\": \"$TENANTACCOUNTID\"}" | python -m
json.tool
```

The response for this example includes a signed URL that is URL-encoded, but it does not include the additional JSON-encoding layer.

```
{
    "apiVersion": "3.0",
    "data":
"https://adfs.example.com/adfs/ls/?SAMLRequest=fZHLbsIwEEV%2FJTuv7...
sSl%2BfQ33cvfwA%3D&RelayState=12345",
    "responseTime": "2018-11-06T16:30:23.355Z",
    "status": "success"
}
```

c. Save the `SAMLRequest` from the response for use in subsequent commands.

```
export SAMLREQUEST='fZHLbsIwEEV%2FJTuv7...sSl%2BfQ33cvfwA%3D'
```

d. Get a full URL that includes the client request ID from AD FS.

One option is to request the login form using the URL from the previous response.

```
curl "https://$AD_FS_ADDRESS/adfs/ls/?SAMLRequest=
$SAMLREQUEST&RelayState=$TENANTACCOUNTID" | grep 'form method="post"
id="loginForm"'
```

The response includes the client request ID:

```
<form method="post" id="loginForm" autocomplete="off"
novalidate="novalidate" onKeyPress="if (event && event.keyCode == 13)
Login.submitLoginRequest();" action="/adfs/ls/?
SAMLRequest=fZHRToMwFIZfhb...UJikvo77sXPw%3D%3D&RelayState=12345&clie
nt-request-id=00000000-0000-0000-ee02-0080000000de" >
```

e. Save the client request ID from the response.

```
export SAMLREQUESTID='00000000-0000-0000-ee02-0080000000de'
```

f. Send your credentials to the form action from the previous response.

```
curl -X POST "https://$AD_FS_ADDRESS
/adfs/ls/?SAMLRequest=$SAMLREQUEST&RelayState=$TENANTACCOUNTID&client
-request-id=$SAMLREQUESTID" \
--data "UserName=$SAMLUSER@$SAMLDOMAIN&Password=
$SAMLPASSWORD&AuthMethod=FormsAuthentication" --include
```

AD FS returns a 302 redirect, with additional information in the headers.

> If multi-factor authentication (MFA) is enabled for your SSO system, the form post will also contain the second password or other credentials.

```
HTTP/1.1 302 Found
Content-Length: 0
Content-Type: text/html; charset=utf-8
Location:
https://adfs.example.com/adfs/ls/?SAMLRequest=fZHRToMwFIZfhb...UJikvo
77sXPw%3D%3D&RelayState=12345&client-request-id=00000000-0000-0000-
ee02-0080000000de
Set-Cookie: MSISAuth=AAEAADAvsHpXk6ApV...pmP0aEiNtJvWY=; path=/adfs;
HttpOnly; Secure
Date: Tue, 06 Nov 2018 16:55:05 GMT
```

g. Save the `MSISAuth` cookie from the response.

```
export MSISAuth='AAEAADAvsHpXk6ApV...pmP0aEiNtJvWY='
```

h. Send a GET request to the specified location with the cookies from the authentication POST.

```
curl "https://$AD_FS_ADDRESS/adfs/ls/?SAMLRequest=
$SAMLREQUEST&RelayState=$TENANTACCOUNTID&client-request-
id=$SAMLREQUESTID" \
--cookie "MSISAuth=$MSISAuth" --include
```

The response headers will contain AD FS session information for later logout usage, and the response body contains the SAMLResponse in a hidden form field.

```
HTTP/1.1 200 OK
Cache-Control: no-cache,no-store
Pragma: no-cache
Content-Length: 5665
Content-Type: text/html; charset=utf-8
Expires: -1
Server: Microsoft-HTTPAPI/2.0
P3P: ADFS doesn't have P3P policy, please contact your site's admin
for more details
Set-Cookie:
SamlSession=a3dpbnRlcnMtUHJpbWFyeS1BZG1pbi0xNzgmRmFsc2Umcng4NnJDZmFKV
XFxVWx3bkl1MnFuUSUzZCUzZCYmJiYmXzE3MjAyZTA5LThmMDgtNDRkZC04Yzg5LTQ3ND
UxYzA3ZjkzYw==; path=/adfs; HttpOnly; Secure
Set-Cookie: MSISAuthenticated=MTEvNy8yMDE4IDQ6MzI6NTkgUE0=;
path=/adfs; HttpOnly; Secure
Set-Cookie: MSISLoopDetectionCookie=MjAxOC0xMS0wNzoxNjozMjo1OVpcMQ==;
path=/adfs; HttpOnly; Secure
Date: Wed, 07 Nov 2018 16:32:59 GMT

<form method="POST" name="hiddenform"
action="https://storagegrid.example.com:443/api/saml-response">
  <input type="hidden" name="SAMLResponse"
value="PHNhbWxwOlJlc3BvbnN...1scDpSZXNwb25zZT4=" /><input
type="hidden" name="RelayState" value="12345" />
```

i. Save the `SAMLResponse` from the hidden field:

```
export SAMLResponse='PHNhbWxwOlJlc3BvbnN...1scDpSZXNwb25zZT4='
```

j. Using the saved `SAMLResponse`, make a StorageGRID `/api/saml-response` request to generate a StorageGRID authentication token.

For `RelayState`, use the tenant account ID or use 0 if you want to sign in to the Grid Management API.

```
curl -X POST "https://$STORAGEGRID_ADDRESS:443/api/saml-response" \
  -H "accept: application/json" \
  --data-urlencode "SAMLResponse=$SAMLResponse" \
  --data-urlencode "RelayState=$TENANTACCOUNTID" \
  | python -m json.tool
```

The response includes the authentication token.

```
{
    "apiVersion": "3.0",
    "data": "56eb07bf-21f6-40b7-af0b-5c6cacfb25e7",
    "responseTime": "2018-11-07T21:32:53.486Z",
    "status": "success"
}
```

k. Save the authentication token in the response as `MYTOKEN`.

```
export MYTOKEN="56eb07bf-21f6-40b7-af0b-5c6cacfb25e7"
```

You can now use `MYTOKEN` for other requests, similar to how you would use the API if SSO was not being used.

Sign out of the API if single sign-on is enabled

If single sign-on (SSO) has been enabled, you must issue a series of API requests to sign out of the Grid Management API or the Tenant Management API. These instructions apply if you are using Active Directory as the SSO identity provider

About this task

If required, you can sign out of the StorageGRID API by logging out from your organization's single logout page. Or, you can trigger single logout (SLO) from StorageGRID, which requires a valid StorageGRID bearer token.

Steps

1. To generate a signed logout request, pass `cookie "sso=true"` to the SLO API:

```
curl -k -X DELETE "https://$STORAGEGRID_ADDRESS/api/v3/authorize" \
-H "accept: application/json" \
-H "Authorization: Bearer $MYTOKEN" \
--cookie "sso=true" \
| python -m json.tool
```

A logout URL is returned:

```
{
    "apiVersion": "3.0",
    "data":
"https://adfs.example.com/adfs/ls/?SAMLRequest=fZDNboMwEIRfhZ...HcQ%3D%3
D",
    "responseTime": "2018-11-20T22:20:30.839Z",
    "status": "success"
}
```

2. Save the logout URL.

```
export LOGOUT_REQUEST
='https://adfs.example.com/adfs/ls/?SAMLRequest=fZDNboMwEIRfhZ...HcQ%3D%
3D'
```

3. Send a request to the logout URL to trigger SLO and to redirect back to StorageGRID.

```
curl --include "$LOGOUT_REQUEST"
```

The 302 response is returned. The redirect location is not applicable to API-only logout.

```
HTTP/1.1 302 Found
Location: https://$STORAGEGRID_ADDRESS:443/api/saml-
logout?SAMLResponse=fVLLasMwEPwVo7ss%...%23rsa-sha256
Set-Cookie: MSISSignoutProtocol=U2FtbA==; expires=Tue, 20 Nov 2018
22:35:03 GMT; path=/adfs; HttpOnly; Secure
```

4. Delete the StorageGRID bearer token.

Deleting the StorageGRID bearer token works the same way as without SSO. If `cookie "sso=true"` is not provided, the user is logged out of StorageGRID without affecting the SSO state.

```
curl -X DELETE "https://$STORAGEGRID_ADDRESS/api/v3/authorize" \
-H "accept: application/json" \
-H "Authorization: Bearer $MYTOKEN" \
--include
```

A `204 No Content` response indicates the user is now signed out.

```
HTTP/1.1 204 No Content
```

Use the API if single sign-on is enabled (Azure)

If you have configured and enabled single sign-on (SSO) and you use Azure as the SSO provider, you can use two example scripts to obtain an authentication token that is valid for the Grid Management API or the Tenant Management API.

Sign in to the API if Azure single sign-on is enabled

These instructions apply if you are using Azure as the SSO identity provider

Before you begin

- You know the SSO email address and password for a federated user who belongs to a StorageGRID user group.
- If you want to access the Tenant Management API, you know the tenant account ID.

About this task

To obtain an authentication token, you can use the following example scripts:

- The `storagegrid-ssoauth-azure.py` Python script
- The `storagegrid-ssoauth-azure.js` Node.js script

Both scripts are located in the StorageGRID installation files directory (`./rpms` for Red Hat Enterprise Linux or CentOS, `./debs` for Ubuntu or Debian, and `./vsphere` for VMware).

To write your own API integration with Azure, see the `storagegrid-ssoauth-azure.py` script. The Python script makes two requests to StorageGRID directly (first to get the SAMLRequest, and later to get the authorization token), and also calls the Node.js script to interact with Azure to perform the SSO operations.

SSO operations can be executed using a series of API requests, but doing so is not straightforward. The Puppeteer Node.js module is used to scrape the Azure SSO interface.

If you have a URL-encoding issue, you might see the error: `Unsupported SAML version`.

Steps

1. Install the required dependencies, as follows:
   a. Install Node.js (see https://nodejs.org/en/download/).
   b. Install the required Node.js modules (puppeteer and jsdom):

      ```
      npm install -g <module>
      ```

2. Pass the Python script to the Python interpreter to run the script.

   The Python script will then call the corresponding Node.js script to perform the Azure SSO interactions.

3. When prompted, enter values for the following arguments (or pass them in using parameters):
   - The SSO email address used to sign in to Azure
   - The address for StorageGRID
   - The tenant account ID, if you want to access the Tenant Management API

4. When prompted, enter the password and be prepared to provide an MFA authorization to Azure if requested.

```
c:\Users\user\Documents\azure_sso>py storagegrid-azure-ssoauth.py --sso-email-address user@my-domain.com
--sg-address storagegrid.examp.e.com --tenant-account-id 0
Enter the user's SSO password:
*******************************************************************************

Watch for and approve a 2FA authorization request
*******************************************************************************
StorageGRID Auth Token: {'responseTime': '2021-10-04T21:30:48.807Z', 'status': 'success', 'apiVersion':
'3.4', 'data': '4807d93e-a3df-48f2-9680-906cd255979e'}
```

> The script assumes MFA is done using Microsoft Authenticator. You might need to modify the script to support other forms of MFA (such as entering a code received in a text message).

The StorageGRID authorization token is provided in the output. You can now use the token for other requests, similar to how you would use the API if SSO was not being used.

Use the API if single sign-on is enabled (PingFederate)

If you have configured and enabled single sign-on (SSO) and you use PingFederate as the SSO provider, you must issue a series of API requests to obtain an authentication token that is valid for the Grid Management API or the Tenant Management API.

Sign in to the API if single sign-on is enabled

These instructions apply if you are using PingFederate as the SSO identity provider

Before you begin

- You know the SSO username and password for a federated user who belongs to a StorageGRID user group.
- If you want to access the Tenant Management API, you know the tenant account ID.

About this task

To obtain an authentication token, you can use one of the following examples:

- The `storagegrid-ssoauth.py` Python script, which is located in the StorageGRID installation files directory (`./rpms` for Red Hat Enterprise Linux or CentOS, `./debs` for Ubuntu or Debian, and `./vsphere` for VMware).
- An example workflow of curl requests.

  The curl workflow might time out if you perform it too slowly. You might see the error: `A valid SubjectConfirmation was not found on this Response`.

  > The example curl workflow does not protect the password from being seen by other users.

  If you have a URL-encoding issue, you might see the error: `Unsupported SAML version`.

Steps

1. Select one of the following methods to obtain an authentication token:
   - Use the `storagegrid-ssoauth.py` Python script. Go to step 2.
   - Use curl requests. Go to step 3.
2. If you want to use the `storagegrid-ssoauth.py` script, pass the script to the Python interpreter and run the script.

   When prompted, enter values for the following arguments:

   - The SSO method. You can enter any variation of “pingfederate” (PINGFEDERATE, pingfederate, and so on).
   - The SSO username
   - The domain where StorageGRID is installed. This field is not used for PingFederate. You can leave it blank or enter any value.
   - The address for StorageGRID
   - The tenant account ID, if you want to access the Tenant Management API.

   ```
   python3 storagegrid-ssoauth.py
   sso_method: pingfederate
   saml_user: my-sso-username
   saml_domain:
   sg_address: storagegrid.example.com
   tenant_account_id: 12345
   Enter the user's SAML password:
   ********************************************************************************

   ********************************************************************************
   StorageGRID Auth Token: 56eb07bf-21f6-40b7-afob-5c6cacfb25e7
   ```

   The StorageGRID authorization token is provided in the output. You can now use the token for other requests, similar to how you would use the API if SSO was not being used.

3. If you want to use curl requests, use the following procedure.
   a. Declare the variables needed to sign in.

   ```
   export SAMLUSER='my-sso-username'
   export SAMLPASSWORD='my-password'
   export TENANTACCOUNTID='12345'
   export STORAGEGRID_ADDRESS='storagegrid.example.com'
   ```

   To access the Grid Management API, use 0 as `TENANTACCOUNTID`.

   b. To receive a signed authentication URL, issue a POST request to `/api/v3/authorize-saml`, and remove the additional JSON encoding from the response.

   This example shows a POST request for a signed authentication URL for TENANTACCOUNTID. The results will be passed to python -m json.tool to remove the JSON encoding.

```
curl -X POST "https://$STORAGEGRID_ADDRESS/api/v3/authorize-saml" \
  -H "accept: application/json" -H  "Content-Type: application/json"
\
  --data "{\"accountId\": \"$TENANTACCOUNTID\"}" | python -m
json.tool
```

The response for this example includes a signed URL that is URL-encoded, but it does not include the additional JSON-encoding layer.

```
{
    "apiVersion": "3.0",
    "data": "https://my-pf-baseurl/idp/SSO.saml2?...",
    "responseTime": "2018-11-06T16:30:23.355Z",
    "status": "success"
}
```

c. Save the `SAMLRequest` from the response for use in subsequent commands.

```
export SAMLREQUEST="https://my-pf-baseurl/idp/SSO.saml2?..."
```

d. Export the response and cookie, and echo the response:

```
RESPONSE=$(curl -c - "$SAMLREQUEST")
```

```
echo "$RESPONSE" | grep 'input type="hidden" name="pf.adapterId"
id="pf.adapterId"'
```

e. Export the 'pf.adapterId' value, and echo the response:

```
export ADAPTER='myAdapter'
```

```
echo "$RESPONSE" | grep 'base'
```

f. Export the 'href' value (remove the trailing slash /), and echo the response:

```
export BASEURL='https://my-pf-baseurl'
```

```
echo "$RESPONSE" | grep 'form method="POST"'
```

g. Export the 'action' value:

```
export SSOPING='/idp/.../resumeSAML20/idp/SSO.ping'
```

h. Send cookies along with credentials:

```
curl -b <(echo "$RESPONSE") -X POST "$BASEURL$SSOPING" \
--data "pf.username=$SAMLUSER&pf.pass=
$SAMLPASSWORD&pf.ok=clicked&pf.cancel=&pf.adapterId=$ADAPTER"
--include
```

i. Save the `SAMLResponse` from the hidden field:

```
export SAMLResponse='PHNhbWxwOlJlc3BvbnN...1scDpSZXNwb25zZT4='
```

j. Using the saved `SAMLResponse`, make a StorageGRID `/api/saml-response` request to generate a StorageGRID authentication token.

   For `RelayState`, use the tenant account ID or use 0 if you want to sign in to the Grid Management API.

```
curl -X POST "https://$STORAGEGRID_ADDRESS:443/api/saml-response" \
  -H "accept: application/json" \
  --data-urlencode "SAMLResponse=$SAMLResponse" \
  --data-urlencode "RelayState=$TENANTACCOUNTID" \
  | python -m json.tool
```

   The response includes the authentication token.

```
{
    "apiVersion": "3.0",
    "data": "56eb07bf-21f6-40b7-af0b-5c6cacfb25e7",
    "responseTime": "2018-11-07T21:32:53.486Z",
    "status": "success"
}
```

k. Save the authentication token in the response as `MYTOKEN`.

```
export MYTOKEN="56eb07bf-21f6-40b7-af0b-5c6cacfb25e7"
```

You can now use `MYTOKEN` for other requests, similar to how you would use the API if SSO was not being used.

Sign out of the API if single sign-on is enabled

If single sign-on (SSO) has been enabled, you must issue a series of API requests to sign out of the Grid Management API or the Tenant Management API. These instructions apply if you are using PingFederate as the SSO identity provider

About this task

If required, you can sign out of the StorageGRID API by logging out from your organization's single logout page. Or, you can trigger single logout (SLO) from StorageGRID, which requires a valid StorageGRID bearer token.

Steps

1. To generate a signed logout request, pass `cookie "sso=true"` to the SLO API:

```
curl -k -X DELETE "https://$STORAGEGRID_ADDRESS/api/v3/authorize" \
-H "accept: application/json" \
-H "Authorization: Bearer $MYTOKEN" \
--cookie "sso=true" \
| python -m json.tool
```

A logout URL is returned:

```
{
    "apiVersion": "3.0",
    "data": "https://my-ping-
url/idp/SLO.saml2?SAMLRequest=fZDNboMwEIRfhZ...HcQ%3D%3D",
    "responseTime": "2021-10-12T22:20:30.839Z",
    "status": "success"
}
```

2. Save the logout URL.

```
export LOGOUT_REQUEST='https://my-ping-
url/idp/SLO.saml2?SAMLRequest=fZDNboMwEIRfhZ...HcQ%3D%3D'
```

3. Send a request to the logout URL to trigger SLO and to redirect back to StorageGRID.

```
curl --include "$LOGOUT_REQUEST"
```

The 302 response is returned. The redirect location is not applicable to API-only logout.

```
HTTP/1.1 302 Found
Location: https://$STORAGEGRID_ADDRESS:443/api/saml-
logout?SAMLResponse=fVLLasMwEPwVo7ss%...%23rsa-sha256
Set-Cookie: PF=QoKs...SgCC; Path=/; Secure; HttpOnly; SameSite=None
```

4. Delete the StorageGRID bearer token.

   Deleting the StorageGRID bearer token works the same way as without SSO. If `cookie "sso=true"` is not provided, the user is logged out of StorageGRID without affecting the SSO state.

```
curl -X DELETE "https://$STORAGEGRID_ADDRESS/api/v3/authorize" \
-H "accept: application/json" \
-H "Authorization: Bearer $MYTOKEN" \
--include
```

   A `204 No Content` response indicates the user is now signed out.

```
HTTP/1.1 204 No Content
```

Copyright information

Copyright © 2024 NetApp, Inc. All Rights Reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means—graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system—without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

LIMITED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (b)(3) of the Rights in Technical Data -Noncommercial Items at DFARS 252.227-7013 (FEB 2014) and FAR 52.227-19 (DEC 2007).

Data contained herein pertains to a commercial product and/or commercial service (as defined in FAR 2.101) and is proprietary to NetApp, Inc. All NetApp technical data and computer software provided under this Agreement is commercial in nature and developed solely at private expense. The U.S. Government has a non-exclusive, non-transferrable, nonsublicensable, worldwide, limited irrevocable license to use the Data only in connection with and in support of the U.S. Government contract under which the Data was delivered. Except as provided herein, the Data may not be used, disclosed, reproduced, modified, performed, or displayed without the prior written approval of NetApp, Inc. United States Government license rights for the Department of Defense are limited to those rights identified in DFARS clause 252.227-7015(b) (FEB 2014).

Trademark information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.